

CM1210 - Assessment 2 - Data Structures and Algorithms in Java REPORT2

Stack Class Design and Implementation

I created a custom generic stack rather than using Java's built-in classes. Making it a generic `Stack<T>` allows it to store different types of data, particularly `EditAction` objects in this case.

It is based on a singly linked list, with an inner node class that stores the data and a reference to the next node in the stack. The stack keeps track of a single reference called `top`, which represents the most recent element.

When pushing, a new node is created and placed on `top`. When popping, the `top` value is returned and the reference moves to the next node. There is a check in places called `isEmpty`, to ensure that the stack is not empty before popping, preventing errors.

This design is efficient because all operations run in constant time $O(1)$, and satisfies the requirement to avoid built-in collections.

Use of Stacks for Undo and Redo Functionality

Two stacks are used, one for undo operations and one for redo operations. Every user action (typing and deleting) is stored as an `EditAction` object.

When a new action is performed it is pushed onto the undo stack and the redo stack is cleared. This ensures that redo history does not conflict with new actions

In order to undo an action, the most recent action is removed from the undo stack, reversed, and placed onto the redo stack. In order to redo, the last undone action is taken from the redo stack, reapplied, and pushed back onto the undo stack.

This works because stacks follow a Last-In, First-Out (LIFO) principle, meaning the most recent action is always processed first, which matches how

undo/redo systems behave.

Evaluation of Alternative Data Structures

I could use a doubly linked list or a deque, as they allow movement in both directions and can store a full history of actions. However, these structures add unnecessary complexity and overhead for this task. Since undo and redo only require access to the most recent actions, a stack is more suitable. It provides a simpler implementation and directly supports the required LIFO behaviour.